

Конспект по теме: "Ввод и вывод данных. Параметры sep и end."

1. Вывод данных: функция `print()`

Функция `print()` используется для вывода данных на экран. Она может принимать несколько аргументов, разделяя их пробелами по умолчанию.

- Формат: `print(значение1, значение2, ..., значениеN)`.

Пример:

```
print("Имя:", "Alice", "Возраст:", 25)
```

- Особенности:
 - Можно выводить несколько значений разных типов (строки, числа и т.д.).
 - Функция автоматически добавляет пробелы между аргументами и перенос строки в конце.

2. Дополнительные возможности функции `print()`

2.1 Специальные символы

Специальные символы используются для форматирования текста.

- `\n` - Перенос строки (новая строка)

Используется для перехода на новую строку.

Пример:

```
print("Привет\nМир")
```

Вывод:

```
Привет
Мир
```

- `\t` — Табуляция (отступ)

Добавляет горизонтальный отступ (табуляцию), эквивалентен нескольким пробелам (обычно 4 или 8).

Пример:

```
print("Колонка 1\tКолонка 2")
```

Вывод:

```
Колонка 1    Колонка 2
```

- `\\` — Обратный слэш

Используется для вывода символа обратного слэша `\`.

Пример:

```
print("Путь к файлу: C:\\Users\\Имя")
```

Вывод:

```
Путь к файлу: C:\Users\Имя
```

- **\' — Одинарная кавычка**

Используется для вставки одинарной кавычки в строку.

Пример:

```
print('Это мой \'кот\')
```

Вывод:

```
Это мой 'кот'
```

- **\" — Двойная кавычка**

Используется для вставки двойной кавычки в строку.

Пример:

```
print("Она сказала: \"Привет!\")
```

Вывод:

```
Она сказала: "Привет!"
```

- **\r — Возврат каретки**

Возвращает каретку (курсор) в начало текущей строки без перехода на новую строку. Текст, который будет выведен после `\r`, заменяет текст до этого символа.

Пример:

```
print("Привет\rМир")
```

Вывод:

```
Мирет
```

- **\b — Забой (backspace)**

Удаляет предыдущий символ (в символьном смысле).

Пример:

```
print("Hellooo\b!")
```

Вывод:

```
Helloo!
```

- **\f — Перевод страницы (form feed)**

Переводит текст на новую страницу. В консольном выводе это редко используется, но полезно для управления печатными устройствами.

Пример:

```
print("Текст до страницы\fТекст после страницы")
```

- **\a — Звуковой сигнал (bell)**

Издаёт звуковой сигнал (бип), если поддерживается терминалом или устройством.

Пример:

```
print("\a")
```

Вывод: Проигрывается звуковой сигнал.

- **\v — Вертикальная табуляция**

Добавляет вертикальный отступ, похожий на горизонтальную табуляцию, но вниз по строке.

Пример:

```
print("Текст до\vТекст после")
```

Вывод:

```
Текст до
      Текст после
```

- **\0 — Нулевой символ (null)**

Представляет символ с кодом 0. Обычно используется в низкоуровневом программировании, но редко применяется в Python.

Пример:

```
print("Текст\0Текст")
```

Эти специальные символы полезны для форматирования и управления выводом в Python.

2.2 Форматирование строк с помощью **f-string**

f-string позволяет вставлять значения переменных внутрь строки.

Формат: `f"текст {переменная} текст"`.

Пример:

```
name = "Alice"
age = 25
print(f"Меня зовут {name}, мне {age} лет.")
```

Вывод:

```
Меня зовут Alice, мне 25 лет.
```

2.3 Округление чисел

Округление чисел в строках можно выполнять с помощью **f-string**.

- Формат: `f"{число:.nf}"`, где `n` — количество знаков после запятой.

Пример:

```
number = 3.14159
print(f"Округленное значение: {number:.2f}")
```

Вывод:

```
Округленное значение: 3.14
```

2.4 Многострочные строки

Тройные кавычки позволяют выводить многострочные строки.

Пример:

```
print("""Первая строка.
Вторая строка.
Третья строка.""")
```

Вывод:

```
Первая строка.
Вторая строка.
Третья строка.
```

Заключение

- Функции `input()` и `print()` — основные инструменты для взаимодействия пользователя с программой.
- Параметры `sep` и `end` позволяют гибко форматировать вывод данных.
- Специальные символы и форматирование строк с помощью `f-string` расширяют возможности работы с текстом и числами.